

# Attivare il collegamento ZeDMD-WiFi

---

## Attivare il collegamento ZeDMD-WiFi

---

Guida complementare al *Manuale completo*. Riguarda il momento in cui il DMD, già funzionante di suo, deve iniziare a ricevere le immagini da Batocera o da Visual Pinball.

Software di riferimento: **kWGillo DMD Server 2.0** · <https://github.com/kWGillo/zedmd-pi>

---

### 1. Che cosa significa “ZeDMD-WiFi” in questo progetto

ZeDMD nasce come dispositivo su ESP32. Nella versione WiFi il client — Batocera, [dmdserver](#), dmd-extensions, Visual Pinball — non cerca una porta seriale: cerca un indirizzo IP sulla rete e gli parla con un protocollo suo.

Il DMD Controller **implementa quel protocollo**. Dal punto di vista del client non c'è differenza: interroga un indirizzo, riceve le caratteristiche del dispositivo e comincia a inviare i frame. Che dall'altra parte ci sia un ESP32 o un Raspberry Pi non lo riguarda.

Quindi “attivare il WiFi di ZeDMD” qui vuol dire due cose, in quest'ordine:

1. il Raspberry deve essere raggiungibile sulla rete (sezione 2)
2. il client deve sapere a quale indirizzo parlare (sezioni 4 e 5)

Non c'è nessun interruttore “WiFi” da attivare sul DMD: il servizio è già in ascolto dal momento in cui parte.

---

## 2. Il Raspberry sulla rete Wi-Fi

### 2.1 Verificare la situazione attuale

Via SSH sul Raspberry:

```
hostname -I
```

```
nmcli device status
```

Il primo restituisce l'indirizzo IP — è quello che servirà al client. Il secondo mostra lo stato delle interfacce: `wlan0` deve risultare **connected**.

### 2.2 Collegarsi a una rete diversa

Le versioni recenti di Raspberry Pi OS usano NetworkManager. Per vedere le reti disponibili:

```
nmcli device wifi list
```

Per collegarsi:

```
sudo nmcli device wifi connect "NOME_RETE" password "PASSWORD"
```

La rete viene memorizzata e riagganciata da sola a ogni avvio. In alternativa, con interfaccia testuale:

```
sudo raspi-config
```

*System Options* → *Wireless LAN*.

---

```
Se il Wi-Fi risultasse bloccato ( rftkill: blocked ), sbloccalo con sudo rftkill unblock wifi.
```

Su immagini più vecchie, che non usano NetworkManager, la rete si configura in `/etc/wpa_supplicant/wpa_supplicant.conf`. Se `nmcli` non esiste, sei in quel caso.

## 2.3 Banda e collocazione

La **Pi Zero 2 W** ha solo **Wi-Fi a 2,4 GHz**. Se il router separa le due bande con SSID diversi, va indicata quella a 2,4. Pi 3B+ e Pi 4 supportano anche i 5 GHz.

Il DMD sta dentro un cabinato di legno, spesso con lamiere e trasformatori vicini: se il segnale è debole i frame arrivano a scatti. Verifica la qualità:

```
nmcli -f IN-USE,SSID,SIGNAL,CHAN device wifi list | head -5
```

Sotto il 50% conviene avvicinare il router o spostare l'antenna.

## 2.4 Indirizzo IP fisso — fortemente consigliato

Il client ZeDMD memorizza l'indirizzo che gli indichi. Se il router assegna al Raspberry un IP diverso dopo un riavvio, **il DMD smette di funzionare senza alcun messaggio d'errore**: il client continua a cercare un indirizzo dove non c'è più nessuno.

Il modo pulito è la **prenotazione DHCP sul router**: si associa l'indirizzo MAC del Raspberry a un IP fisso. Il MAC si legge così:

```
cat /sys/class/net/wlan0/address
```

Poi, nella pagina di amministrazione del router, si cerca la sezione DHCP e si aggiunge la riserva. È preferibile all'IP statico configurato sul Raspberry, perché resta gestito in un solo posto e non rischia di collidere con altri dispositivi.

## 3. Le porte in gioco

| Porta | Uso                                                               | Note                                         |
|-------|-------------------------------------------------------------------|----------------------------------------------|
| 80    | handshake: il client chiede al dispositivo le sue caratteristiche | <b>cablata nel client</b> , non modificabile |
| 3333  | flusso dei frame DMD, TCP e UDP                                   | <code>zedmd.stream_port</code>               |
| 8080  | interfaccia web del DMD Controller                                | non usata dal protocollo                     |

Al client si indica **soltanto l'indirizzo IP**: le porte le conosce già.

**Perché l'handshake non passa da Flask.** Il client legge la risposta HTTP con una sola `recv()` e si ferma appena riceve meno di 1024 byte. Se intestazioni e corpo arrivano in due pacchetti distinti — come fa Flask — legge un corpo vuoto, non riconosce il trasporto TCP e ripiega su UDP, apparentemente collegandosi ma senza mostrare nulla. Per questo la porta 80 è servita da un socket server dedicato che scrive tutto in una sola operazione, e l'interfaccia web vive sulla 8080.

## 4. Batocera

È il caso verificato sul campo.

### 4.1 File di configurazione

Batocera pilota i DMD reali attraverso `dmdserver`. Nell'interfaccia non esiste un campo per l'indirizzo di rete: sta in un file. Via SSH sulla macchina Batocera, come `root`:

```
cat > /userdata/system/configs/dmdserver/config.ini << 'EOF'
[DMDServer]
AltColor = 1

[ZeDMD]
Enabled = 0

[ZeDMD-WiFi]
Enabled = 1
WiFiAddr = 192.168.0.XXX
EOF
```

Sostituisci `192.168.0.XXX` con l'indirizzo del Raspberry, quello ottenuto al punto 2.1.

Due righe meritano una spiegazione:

- `[ZeDMD] Enabled = 0` disattiva la ricerca del dispositivo su porta seriale. Con il collegamento di rete attivo non serve, e lasciarla accesa può creare conflitti.
- `AltColor = 1` abilita le colorazioni alternative dei giochi, quando disponibili. È indipendente dal collegamento: puoi metterlo a 0 senza conseguenze sulla connessione.

## 4.2 Attivare il servizio

Nel menu di Batocera, attiva **DMD reale**.

Non confonderlo con **DMD Web**, che è il simulatore su browser: è una cosa diversa e non parla con il tuo pannello.

Il servizio si chiama `dmd_real`, e da riga di comando si attiva così:

```
batocera-services enable dmd_real
batocera-services start dmd_real
```

**Verifica che sia partito davvero**, perché è il punto in cui si perde più tempo:

```
ps aux | grep dmdserver | grep -v grep
```

Deve comparire un processo `dmdserver` con l'argomento `-c` `/userdata/system/configs/dmdserver/config.ini`. Se non compare niente, il file di configurazione che hai appena scritto non lo sta leggendo nessuno: il `config.ini` da solo non avvia nulla, e il Raspberry resterà in ascolto senza mai vedere un client. Se provi a lanciare `dmdserver` a mano senza `-c`, muore dopo pochi secondi con codice 2: senza configurazione non trova display e termina, che è il suo comportamento normale.

Due trappole viste sul campo:

- **Installare Pixelcade** aggiunge una propria gestione del DMD e la chiave `dmd.pixelcade.dmdserver` in `batocera.conf`. Quella chiave la legge solo lo script di Pixelcade: non è l'interruttore di `dmd_real`, e cambiarla non avvia niente.
- **Cambiando il Raspberry cambia l'indirizzo IP.** `WiFiAddr` punta ancora al vecchio, e il sintomo è identico a "il servizio non parte". Il modo più rapido per distinguerli è provare l'handshake dal cabinato:

```
curl -s http://192.168.0.XXX/handshake; echo
```

Se risponde con i 22 campi separati da `|`, la rete e il Raspberry sono a posto e il problema è solo di chi deve avviare `dmdserver`.

## 4.3 Un limite di EmulationStation, non del DMD

Tenendo premuto il tasto di scorrimento, l'immagine sul pannello **non si aggiorna**, e non si aggiorna nemmeno quando rilasci: resta quella dell'elemento da cui sei partito. Basta un tocco in più per allinearla.

Non è un difetto del collegamento. EmulationStation apre una connessione nuova verso `dmdserver` a ogni cambio di selezione, ma durante la ripetizione automatica del tasto non ne apre nessuna, e non ne apre una finale al rilascio: il pannello mostra correttamente l'ultima immagine che gli è stata mandata. Lo si verifica con `dmdserver -c ... -v`, che durante la pressione non stampa nulla.

#### 4.4 Dopo ogni aggiornamento del DMD Controller

**Riavvia Batocera.** Il client tiene in memoria lo stato della connessione e la contabilità delle zone dell'immagine già inviate. Quando il servizio sul Raspberry si riavvia, quello stato non è più valido: senza un riavvio del client il display può restare fermo sull'ultimo contenuto o aggiornarsi solo in parte.

---

## 5. Visual Pinball e dmd-extensions (Windows)

Se in futuro vorrai collegare anche un PC Windows con Visual Pinball, il riferimento è `DmdDevice.ini` di `dmd-extensions`.

Le sezioni pertinenti sono `[zedmdwifi]` e `[zedmdhdwifi]`. La chiave dell'indirizzo si chiama `wifi.address` e va **decommentata**, altrimenti il client tenta l'autodiscovery:

```
[zedmdwifi]
enabled = true
wifi.address = 192.168.0.XXX
```

Il DMD Controller dichiara nell'handshake una risoluzione di **256×64**, che corrisponde a quella di ZeDMD HD. Se con `[zedmdwifi]` l'immagine risultasse di dimensione sbagliata, prova la sezione `[zedmdhdwifi]` con le stesse chiavi. Attiva **una sola** delle due.

Ricorda anche di disattivare le altre sezioni di dispositivo (`[zedmd]`, `[pindmd3]`, `[virtualdmd]` e simili) mettendole a `enabled = false`, per non lasciare il client a cercare hardware che non c'è.

---

## 6. Verifica del collegamento

Apri il registro sul Raspberry **prima** di avviare il client:

```
journalctl -u dmd -f
```

Poi fai partire il DMD dal lato client. Deve comparire:

```
[zedmd] client connesso via TCP: ('192.168.0.XXX', ...)
```

Da quel momento il pannello mostra ciò che invia il client, e l'orologio si fa da parte.

Puoi anche interrogare l'handshake a mano, dal Raspberry o da qualsiasi macchina della rete:

```
curl http://192.168.0.XXX/handshake
```

Risponde con una riga di 22 campi separati da `|`. I primi due sono larghezza e altezza: devono essere `256` e `64`.

Nella pagina **Servizi** dell'interfaccia web, la riga di stato di ZeDMD mostra il client collegato e il numero di frame ricevuti. È il modo più rapido per sapere se i dati stanno davvero arrivando.

---

## 7. Risoluzione problemi

| Sintomo                                                                   | Causa probabile                               | Rimedio                                                                                             |
|---------------------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Nel log solo <code>GET /handshake</code> ripetuto, nessuna connessione    | il client non riesce ad aprire il flusso      | verifica che <code>zedmd.http_port</code> sia 80 e <code>web.port</code> sia 8080                   |
| Nessuna richiesta nel log                                                 | indirizzo sbagliato o rete diversa            | ricontrolla <code>hostname -I</code> e l'indirizzo scritto nel client                               |
| Funzionava, poi ha smesso dopo un riavvio del router                      | il Raspberry ha cambiato IP                   | prenotazione DHCP, sezione 2.4                                                                      |
| <code>curl</code> non risponde da un'altra macchina                       | firewall o reti separate (Wi-Fi ospiti, VLAN) | metti client e DMD sulla stessa rete                                                                |
| <code>Address already in use</code> sulla porta 80                        | un altro web server sul Raspberry             | ferma <code>lighttpd</code> , <code>apache2</code> o <code>nginx</code>                             |
| Il display resta sull'ultima immagine dopo un aggiornamento               | stato del client non più valido               | riavvia il client                                                                                   |
| Dopo uno spegnimento brusco di Batocera il display non torna all'orologio | la connessione TCP non è stata chiusa         | attendi: <code>client_timeout</code> (10 s) più <code>grace_seconds</code> (60 s), circa 70 secondi |
| Immagine a scatti solo durante il gioco                                   | segnale Wi-Fi debole                          | sezione 2.3                                                                                         |
| L'orologio si intromette durante le pause di Batocera                     | tempo di grazia troppo corto                  | alza <code>zedmd.grace_seconds</code>                                                               |

## 8. Parametri correlati

Nella configurazione del DMD Controller ( `/etc/dmd/config.json` , oppure dalla pagina Impostazioni):

| Chiave                                   | Default  | Significato                                             |
|------------------------------------------|----------|---------------------------------------------------------|
| <code>zedmd.http_port</code>             | 80       | porta dell'handshake, <b>non modificare</b>             |
| <code>zedmd.stream_port</code>           | 3333     | porta dei frame                                         |
| <code>zedmd.transport</code>             | TCP      | trasporto dichiarato nell'handshake                     |
| <code>zedmd.grace_seconds</code>         | 60       | quanto ZeDMD trattiene il display dopo l'ultimo frame   |
| <code>zedmd.client_timeout</code>        | 10       | silenzio oltre il quale il client è considerato caduto  |
| <code>zedmd.device_name</code>           | ZeDMD-Pi | nome dichiarato al client                               |
| <code>display.sleep_wake_on_zedmd</code> | true     | risveglia il display durante lo Sleep se arrivano frame |

Il trasporto TCP è dichiarato di proposito: evita la frammentazione UDP, che sui dispositivi ZeDMD reali causa problemi durante le raffiche di frame.